Dal desktop al mobile: responsive, PWA, Capacitor

Corso di Frontend Development – Modulo 03

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. Dal desktop al mobile: responsive, PWA, Capacitor
1.1 Il mobile è un altro mondo
1.2 Responsive design
1.3 Progressive Web App
1.4 Capacitor (e Ionic)
1.5 Confronto

Sommario del modulo

1. Dal desktop al mobile: responsive, PWA, Capacitor
1.1 Il mobile è un altro mondo
1.2 Responsive design
1.3 Progressive Web App
1.4 Capacitor (e Ionic)
1.5 Confronto

Due utenti diversi, sullo stesso telefono

Il cliente: catalogo e carrello

- Arriva da un link o da una ricerca, apre la scheda articolo, aggiunge al carrello, invia l'ordine.
- Rete mobile, fretta, una mano sola.
- Deve caricare in fretta, scorrere bene e avere pulsanti che si prendono al primo colpo.

Il magazziniere: l'area admin

- Gira fra gli scaffali, aggiorna le scorte, fotografa gli articoli nuovi.
- Wi-Fi che cade in magazzino, telefono in una mano e scatola nell'altra.
- Ha bisogno di **fotocamera**, di funzionare **offline** e di bersagli grandi.

Tre livelli, in ordine crescente di "nativo"

1. Web responsive $\rightarrow$ 2. PWA installabile $\rightarrow$ 3. App Capacitor negli store. Li facciamo tutti e tre sulla stessa minicommerce. Il campo `isAdmin` separa i due mondi: il carrello è solo del cliente, a un admin l'API risponde 403.

Cosa cambia su un telefono

- **Schermo piccolo e verticale**: la tabella admin a sei colonne non entra.
- **Touch**: niente hover, dita larghe (bersaglio minimo 44 px), gesti.
- **Rete instabile**: 4G in strada, Wi-Fi che cade in magazzino.
- **Batteria e CPU**: il JavaScript pesante si sente.
- **Interruzioni**: telefonata, notifica, app in background.

- Tastiera virtuale che copre metà schermo, proprio sopra il form dell'articolo o i campi del pagamento.
- Aree "non sicure": notch, barra home, angoli arrotondati.
- Aspettative da app nativa: barra in basso, swipe indietro, splash.
- Due piattaforme con regole diverse (Android, iOS).

Le basi che non si possono sbagliare

```
<!-- index.html del progetto: manca viewport-fit, serve per notch e barra home -->
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
```

```
/* altezza reale dello schermo, barre del browser incluse */
.shell { min-height: 100dvh; padding-bottom: env(safe-area-inset-bottom); }

/* hover solo dove esiste un mouse: su touch l'effetto "resta attaccato" */
@media (hover: hover) { .card:hover { box-shadow: var(--shadow-2); } }

/* bersagli touch */
button, a { min-height: 44px; }
```

- In Tailwind: `min-h-dvh`, `pb-[env(safe-area-inset-bottom)]`; la variante `hover:` si limita con `@media (hover: hover)` nel CSS base.
- Gli `<input>` sotto i 16 px fanno zoomare Safari su iOS: i campi del form articolo e quelli del pagamento devono restare almeno a `text-base`.

Il percorso del cliente è già quasi a posto

```
<!-- CatalogPage, cosi' com'e': una colonna, due da 640px, tre da 1024px -->
<div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
<!-- CartPage: la riga diventa orizzontale solo da 640px in su -->
<Card className="sm:flex-row sm:items-center">
```

- Tailwind è **mobile-first**: le classi senza prefisso valgono sul telefono, `sm:` e `lg:` aggiungono da lì in su. Catalogo, scheda articolo (`/article/:id`) e carrello (`/cart`) sono nati così.
- Quello che resta da sistemare sul telefono:
  - i filtri del catalogo occupano quattro righe: vanno chiusi in un pannello "Filtri" apribile, e `min-w-56` sulla ricerca stringe tutto il resto sotto i 360 px;
  - nel carrello i pulsanti "+" e "−" sono `size="icon"`, cioè 32 px: sotto la soglia del dito;
  - il dialog dell'articolo è alto quanto lo schermo e la tastiera lo copre.
- **Il vero problema è la tabella dell'area admin**: prossima slide.

Una tabella non si mette in un telefono

```
// AdminPage: sei colonne (nome, categoria, marca, prezzo, stock, azioni).
// Sotto i 640 px diventano illeggibili. Due strade:

// A. Scorrimento orizzontale: onesto, veloce, brutto
<div className="overflow-x-auto"><Table>...</Table></div>

// B. Lo stesso dato, due presentazioni
export function ProductAdminList({ products }: { products: Product[] }) {
  const isMobile = useMediaQuery('(max-width: 639px)');
  return isMobile
    ? <div className="divide-y">{products.map(p => <ProductRowCard key={p.id} product={p} />)}</div>
    : <ProductTable products={products} />;
}
```

- La card mobile mostra nome, prezzo e scorte; categoria e marca sotto, più piccole; le azioni diventano uno **swipe** o un menu con tre puntini.
- Stessi dati, stesse mutation, due layout: cambia solo il componente di presentazione.

Container queries: il componente decide da solo

```
<!-- Tailwind v4: @container sul contenitore, @sm: sui figli -->
<div className="@container">
  <article className="flex flex-col @sm:flex-row @sm:items-center gap-3">
    ...
  </article>
</div>
```

- Con le media query un componente sa solo quanto è larga la *finestra*; con le container query sa quanto spazio *ha*.
- Serve esattamente alla card dell'articolo, che vive in tre posti diversi: griglia del catalogo, lista admin, riga del carrello.
- Altre armi moderne: `clamp()` per la tipografia fluida, `grid` con `auto-fit`/`minmax`, il selettore `:has()`.

Pattern di navigazione mobile

- **Barra in basso** con le destinazioni principali: per il cliente Catalogo, Carrello e Profilo, per l'admin Catalogo e Admin. Oggi stanno tutte nell'header (contatore del carrello compreso), raggiungibile solo con il pollice teso.
- **Sheet** dal basso al posto del dialog: il form dell'articolo diventa un pannello che sale, e la tastiera non lo copre.
- Pulsante flottante per l'azione principale: nuovo articolo.
- Filtri in uno sheet, con il numero di filtri attivi sul pulsante.
- Pull-to-refresh: è la `reload()` del vostro hook, attaccata a un gesto.
- Tastiere giuste: `inputMode="decimal"` sul prezzo, `inputMode="numeric"` sulle scorte, `type="email"` su login e registrazione. Nel progetto prezzo e campi della carta lo fanno già.
- Il tasto Indietro di Android deve chiudere lo sheet, non uscire dall'app.

Immagini e prestazioni su rete mobile

- Il problema numero uno di minicommerce su mobile è l'**immagine in base64**: il form admin legge il file con `FileReader.readAsDataURL` e salva la stringa `data:image/...` nella colonna `image_url`. Ogni articolo porta così il suo peso dentro la risposta JSON del catalogo.
- Si vede anche in salita: `express.json()` accetta 100 KB per richiesta, quindi una foto scattata dal telefono fa fallire il salvataggio.
- Soluzione (aperta nel modulo 1, ancora da scrivere): file su disco o su bucket, nel database solo l'URL, poi `<img srcset>` con due o tre dimensioni.
- Formati moderni: **AVIF** e WebP; `loading="lazy"` sotto la piega; `width` e `height` sempre, per non far saltare il layout.
- JavaScript: il code splitting del modulo 1 conta il doppio qui. Obiettivo: meno di 200 KB compressi al primo caricamento del catalogo.
- Misurate su un telefono *vero* di fascia media, non sul vostro portatile.

Provare sul telefono

```
# espone il dev server in rete locale (con npm i flag di Vite vanno dopo --)
npm run dev -- --host       # Vite stampa anche l'indirizzo http://192.168.x.x:5173

# il backend va solo tenuto acceso: app.listen(PORT) senza host ascolta
# gia' su tutte le interfacce, e il proxy di Vite inoltra /api dal PC
```

- Il proxy di Vite inoltra `/api` a `localhost:3001` **dal PC**, quindi funziona anche quando la pagina è aperta dal telefono. Se invece usate `VITE_API_URL` assoluto, lì va messo l'IP del PC.
- **Telefono reale via USB**: `chrome://inspect` su Android, Safari → Sviluppo su iOS con un Mac. DevTools complete, sul dispositivo vero.
- Device Mode di Chrome: buono per il layout, **inaffidabile** per prestazioni, gesti e tastiera.
- Emulatori: Android Studio gira anche su Windows; il simulatore iOS solo su macOS.

Laboratorio 3a

Laboratorio: minicommerce responsive

1. Viewport con `viewport-fit=cover`, `min-h-dvh`, safe area, bersagli da 44 px.
2. Filtri del catalogo dentro uno sheet apribile, con il contatore dei filtri attivi.
3. Tabella admin $\rightarrow$ lista di card sotto i 640 px, con le stesse azioni.
4. Carrello con una mano: pulsanti quantità da 44 px e form di pagamento che la tastiera non copre.
5. Barra di navigazione in basso sotto i 768 px (Catalogo, Carrello, Profilo); il form articolo diventa uno sheet.
6. Provare dal telefono (`npm run dev - -host`) e correggere i tre problemi più evidenti.

Domande di verifica

Perché `100vh` è sbagliato su mobile? Differenza fra media query e container query? Perché iOS zooma sugli input? Cosa rende il catalogo già quasi responsive?

Che cos'è una PWA

- Una web app che il browser può **installare**: icona in home, finestra propria, avvio offline.
- Tre ingredienti: **HTTPS**, un **manifest** (nome, icone, colori), un **Service Worker** (script che intercetta la rete).
- Nessuno store, nessuna approvazione, aggiornamento istantaneo: è "solo" un sito, ma si comporta da app.
- Per minicommerce significa due cose molto concrete: il cliente ha l'icona in home e **rivede il catalogo anche senza rete**; il magazziniere ha la sua app senza passare dallo store.

Manifest e installazione

```
// public/manifest.webmanifest
{
  "name": "minicommerce", "short_name": "minicommerce",
  "start_url": "/", "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff", "theme_color": "#0a0a0a",
  "icons": [
    { "src": "/icons/192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ],
  "shortcuts": [{ "name": "Carrello", "url": "/cart" }]
}
```

- `display: standalone` toglie la barra del browser; `maskable` dà l'icona adattiva su Android.
- Chrome ed Edge propongono l'installazione da soli; su iOS l'utente usa "Aggiungi a Home" da Safari.

vite-plugin-pwa

```
npm install -D vite-plugin-pwa
```

```
// vite.config.ts  (accanto a react() e tailwindcss())
VitePWA({
  registerType: 'prompt',
  manifest: { /* come nella slide precedente */ },
  workbox: {
    globPatterns: ['**/*.{js,css,html,svg,png,woff2}'],       // app shell precachata
    runtimeCaching: [
      { urlPattern: ({ url }) => url.pathname.startsWith('/api/products'),
        handler: 'NetworkFirst',
        options: { cacheName: 'articoli', networkTimeoutSeconds: 3,
                   expiration: { maxEntries: 200, maxAgeSeconds: 86400 } } },
      { urlPattern: ({ url }) => url.pathname.startsWith('/api/categories'),
        handler: 'StaleWhileRevalidate', options: { cacheName: 'categorie' } },
    ],
  },
})
```

Le categorie cambiano una volta al mese: si servono dalla cache e si aggiornano dietro le quinte. Gli articoli cambiano spesso: prima la rete, cache come rete di sicurezza.

Strategie di cache

Strategia	Comportamento	In minicommerce
Precache	scaricato all'installazione, servito dalla cache	JS, CSS, HTML, icone
CacheFirst	cache se c'è, altrimenti rete	immagini degli articoli, font
NetworkFirst	rete, con fallback alla cache	`GET /api/products`
StaleWhileRevalidate	subito la cache, aggiorna dietro	`GET /api/categories`
NetworkOnly	mai cache	login, `/api/cart`, `/api/orders`, scritture admin

- Le scritture non si cachano: si mettono in coda (prossima slide). Nemmeno `GET /api/cart` si cacha: è legato al token di quell'utente.
- Il service worker vive fuori dalla pagina: DevTools → Application per vederlo, aggiornarlo, cancellarlo.

Offline: leggere dalla cache, scrivere in coda

```
// 1. LEGGERE: ci pensa il service worker. GET /api/products in NetworkFirst
//    significa che senza rete il catalogo arriva comunque, dalla cache.

// 2. SCRIVERE: una coda nostra, in localStorage
export async function inviaOppureAccoda(op: Operazione) {
  try { await esegui(op); }
  catch { coda.push(op); salvaCoda(); }      // niente rete: in coda
}
window.addEventListener('online', async () => {
  for (const op of [...coda]) { await esegui(op); togliDallaCoda(op); }
  toast.success('Modifiche inviate');
});

// 3. La UI lo dice
const online = useOnlineStatus();
{!online && <Banner>Sei offline: {coda.length} modifiche in attesa</Banner>}
```

- È lo scenario del magazziniere: conta le scatole senza campo, e quando esce le modifiche partono da sole.
- Vale anche per il cliente: il carrello è **lato server**, quindi offline un `POST /api/cart/items` va in coda. L'ordine invece si fa online: deve vedere subito se è accettato.
- Conflitti: decide l'API. Per le scorte un endpoint `+1/-1` batte il valore assoluto.

Aggiornamenti e prompt di installazione

```
import { useRegisterSW } from 'virtual:pwa-register/react';

export function UpdateToast() {
  const { needRefresh: [needRefresh], updateServiceWorker } = useRegisterSW();
  if (!needRefresh) return null;
  return <Toast action={() => updateServiceWorker(true)}>Nuova versione disponibile</Toast>;
}

// prompt di installazione (Chromium)
window.addEventListener('beforeinstallprompt', (e) => { e.preventDefault(); setInstallEvent(e); });
<Button onClick={() => installEvent.prompt()}>Installa minicommerce</Button>
```

- Un service worker nuovo aspetta che tutte le schede siano chiuse: senza prompt, il negozio resta su una versione vecchia per giorni.
- Mostrate il pulsante “Installa” quando serve davvero: dopo il login, o dopo il primo articolo messo nel carrello. Al visitatore alla prima pagina non interessa.

Limiti delle PWA (soprattutto su iOS)

- Niente accesso a contatti, Bluetooth e NFC su iOS, niente esecuzione continua in background, niente widget.
- iOS può cancellare lo storage di una PWA non usata per qualche settimana: la cache del catalogo sparisce.
- Nessuna presenza negli store, a meno di impacchettarla.
- Push web su iOS solo per PWA *installate* in home; su Android ovunque.
- **PWABuilder** (Microsoft) genera pacchetti per Microsoft Store e Google Play (Trusted Web Activity) partendo dall'URL. Utile se vi serve solo la presenza nello store.

Quindi

PWA per il negozio del cliente (catalogo, scheda articolo, carrello) e per il PC del punto vendita. Per il magazziniere con fotocamera e lettore di codici: Capacitor.

Laboratorio 3b

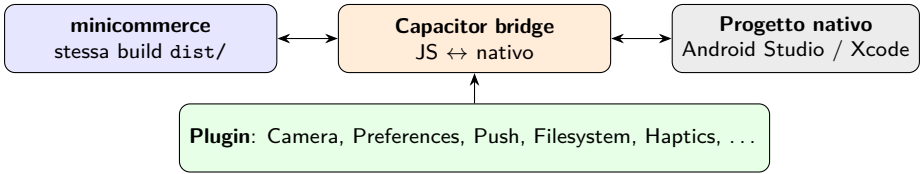
Laboratorio: minicommerce come PWA

1. Manifest, icone (anche maskable), `vite-plugin-pwa` con precache e le regole di runtime per `/api`.
2. Installare l'app dal telefono; in modalità aereo il catalogo si vede ancora, mentre carrello e ordine dicono chiaramente che serve la rete.
3. Coda delle scritture in `localStorage`, invio al ritorno online, banner con il numero di modifiche in attesa.
4. Prompt di aggiornamento; audit Lighthouse: PWA installabile e prestazioni sopra 90.

Domande di verifica

Che differenza c'è fra precache e runtime cache? Perché una nuova versione del service worker non parte subito? Perché le categorie usano StaleWhileRevalidate e gli articoli no?

Capacitor: la webview con i superpoteri



- È l'erede di Cordova, fatto dal team di Ionic. Genera **veri progetti** Android e iOS che potete aprire e modificare.
- Ionic (i componenti UI) è **opzionale**: minicommerce ha già shadcn/ui e Tailwind, e li teniamo.

Aggiungere Capacitor al frontend

```
cd frontend
npm install @capacitor/core && npm install -D @capacitor/cli
npx cap init minicommerce it.its.minicommerce --web-dir dist
npm install @capacitor/android @capacitor/ios
npx cap add android          # crea android/ (progetto Gradle)
npx cap add ios              # crea ios/ (solo su macOS)

npm run build && npx cap sync   # copia dist/ nei progetti nativi e installa i plugin
npx cap open android         # Android Studio -> Run sul telefono o sull'emulatore
```

```
// capacitor.config.ts
const config: CapacitorConfig = {
  appId: 'it.its.minicommerce', appName: 'minicommerce', webDir: 'dist',
  server: { androidScheme: 'https' },   // origine https://localhost
};
```

Le solite due modifiche, più una

```
npm run dev -- --host
npx cap run android --livereload --external     # l'app nativa carica il dev server del PC
```

- **Router**: la pagina è `https://localhost/index.html`, quindi hash router (o `server.url` in sviluppo).
- **API**: oggi `api.ts` usa `baseURL: '/api'` con il proxy di Vite, che nell'app nativa non c'è. Servono `VITE_API_URL` assoluto e **CORS** sul backend per `https://localhost` (Android) e `capacitor://localhost` (iOS). Il `cors()` aperto del progetto li accetta già: in produzione andrà ristretto a questi due valori più il dominio del sito.
- **Piattaforma**: a `lib/piattaforma.ts` si aggiunge il quarto caso.

```
import { Capacitor } from '@capacitor/core';
export const piattaforma = Capacitor.isNativePlatform() ? 'capacitor' : /* ...come prima... */;
```

La fotocamera: finalmente le foto degli articoli

```ts
// npm install @capacitor/camera && npx cap sync
import { Camera, CameraResultType, CameraSource } from '@capacitor/camera';

export async function scattaFotoArticolo(): Promise<File> {
  const photo = await Camera.getPhoto({
    quality: 80, width: 1200,          // ridimensiona PRIMA di inviare
    resultType: CameraResultType.Uri,
    source: CameraSource.Prompt,       // fotocamera o galleria, sceglie l'utente
  });
  const blob = await (await fetch(photo.webPath!)).blob();
  return new File([blob], `articolo.${photo.format}`, { type: blob.type });
}
```

- Sul web lo stesso plugin usa `<input type="file">`: **un solo codice** per le tre piattaforme.
- Va inviata in `multipart` a un endpoint di upload che **oggi non esiste** (lo scrivete nel laboratorio): la base64 del form attuale sfonda i 100 KB di `express.json()`.
- iOS: senza `NSCameraUsageDescription` in `Info.plist` l'app si chiude. Android: permessi nel manifest.

Sessione, tasto Indietro, deep link

```
import { Preferences } from '@capacitor/preferences';   // storage nativo chiave-valore
export const tokenStore = {
  get: async () => (await Preferences.get({ key: 'minicommerce_token' })).value,
  set: (t: string) => Preferences.set({ key: 'minicommerce_token', value: t }),
  clear: () => Preferences.remove({ key: 'minicommerce_token' }),
};

import { App } from '@capacitor/app';
App.addListener('backButton', ({ canGoBack }) => {
  if (sheetAperto) chiudiSheet();
  else if (canGoBack) window.history.back();
  else App.exitApp();
});
App.addListener('appUrlOpen', ({ url }) => {
  // minicommerce://article/7  oppure https://minicommerce.example/article/7
  router.navigate(new URL(url).pathname);
});
```

`Preferences` usa SharedPreferences su Android e UserDefaults su iOS, ma è **asincrono**: l'interceptor di axios legge il token da `localStorage` in modo sincrono, quindi va isolato dietro un `tokenStore`.

Due funzioni che solo l'app nativa può dare

Lettore di codici a barre

- Plugin `@capacitor-mlkit/barcode-scanning`.
- Il magazziniere inquadra il codice EAN e l'articolo si apre.
- Richiede un campo `barcode` nella tabella `products`: una riga nello schema Drizzle e un filtro in più nell'API.

Notifiche push

- `@capacitor/push-notifications` con Firebase Cloud Messaging (e APNs su iOS).
- Al magazziniere: "Tastiera meccanica: scorte esaurite". Al cliente: "Ordine accettato".
- Il backend invia a un token di dispositivo salvato al login.

Rifinitura nativa

`@capacitor/splash-screen`, `status-bar` (colore coerente con il tema), `haptics` (vibrazione quando una scorta va a zero), `keyboard` (ridimensiona la vista quando appare la tastiera).

Build, firma, store

Android

- Android Studio → Generate Signed Bundle (`.aab`), oppure `./gradlew bundleRelease`.
- Keystore: **non perdetelo mai**, senza non si aggiorna più l'app.
- Google Play Console: 25 $ una tantum; test interno → chiuso → produzione.

iOS

- Serve un Mac con Xcode e un account Apple Developer (99 $/anno).
- TestFlight per i tester, poi revisione (1–3 giorni).
- Attenzione: un'app che è solo un catalogo può essere rifiutata come "sito impacchettato". Fotocamera e scanner la rendono un'app vera; il nostro ordine finto, invece, andrebbe completato o tolto.

Laboratorio 3c

Laboratorio: minicommerce su Android (e iOS se avete un Mac)

1. `cap init`, `cap add android`, live reload sul telefono o sull'emulatore.
2. Fotocamera per la foto dell'articolo + endpoint di upload nel backend (multipart, file su disco, URL nel database).
3. `Preferences` per il token; tasto Indietro gestito; deep link `minicommerce://article/:id`.
4. Splash, status bar, haptics; APK di debug installato sul telefono di un compagno.
5. Aggiornare `lib/piattaforma.ts` con il caso Capacitor.

Domande di verifica

Cosa fa `cap sync`? Perché serve l'hash router? Cosa succede su iOS senza la descrizione d'uso della fotocamera? Perché il keystore va custodito?

Responsive vs PWA vs Capacitor vs nativo

	Web responsive	PWA	Capacitor	Nativo
Codice condiviso	100%	100%	95%	0–30%
Store	no	via wrapper	sì	sì
Offline	no	sì	sì	sì
Push	no	sì (limiti iOS)	sì	sì
Fotocamera, scanner	parziale	parziale	plugin	tutto
Prestazioni UI	browser	browser	webview	massime
Feeling nativo	basso	medio	medio/alto	alto
Costo di manutenzione	basso	basso	medio	alto

- Per minicommerce la combinazione sensata è: **PWA per catalogo e carrello** dei clienti, **Capacitor per l'app del magazzino**. Stessa build, due canali.
- Il modulo 5 mostra quando la colonna "Nativo" vince, e perché costa meno di quanto costava.

Dove siamo con minicommerce

Forma	Tecnologia	Stato
Web app	React + Vite	completata (mod. 1)
Desktop	Electron / Tauri	completata (mod. 2)
Mobile web	Responsive + PWA	completata (mod. 3)
Mobile store	Capacitor	completata (mod. 3)
Sito vetrina	SSR/SSG	prossimo modulo
Native	Swift / Kotlin / ...	modulo 5

Un solo repository, una sola logica, quattro modi di installarla. Ora cambiamo completamente problema: **farsi trovare da chi non sa che esistete**.